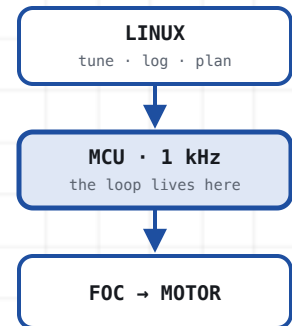


From Sim to Hardware

What to buy, how to split the software, and where the real traps are — opinionated, because you asked for verdicts.

Prices $\approx$ 2026 USD · verify stock before ordering



You've got the physics right from the sim. This is the "turn it into hardware" guide. It's opinionated on purpose. Prices are approximate 2026 USD; a couple of the numbers I was handed were optimistic, so verify current stock.

The single most important sentence here: the fast balance loop lives on the microcontroller, and nothing you put on Linux is allowed inside it. Everything below is downstream of that.

SECTION 01

Architecture – your instinct is right, with one hard constraint

Your proposed layering (Linux app $\rightarrow$ Arduino $\rightarrow$ hardware) is a legitimate, common robotics pattern. It is only wrong in one specific reading of it. The deciding question: **where does the balance loop close?** Your unstable pole is $\lambda \approx 8.45/\text{s}$, an **82 ms doubling time** — the clock you're racing.

Option A – MCU does everything (START HERE)

One microcontroller reads the encoder, runs your Kalman + swing-up + LQR/balance, and drives the motor. Tune by editing constants and reflashing, or a tiny serial shell; log to SD or a telemetry line.

- **Pros:** simplest, most robust, textbook (Cubli, ETH, every serious hobby RWP). Everything hard-real-time on one deterministic chip; jitter in microseconds.
- **Cons:** live tuning is edit-compile-flash; no dashboard unless you build streaming telemetry.
- **Verdict:** this is how you get to first-balance. Build it first, no exceptions.

Option B – balance loop on Linux over USB (DON'T)

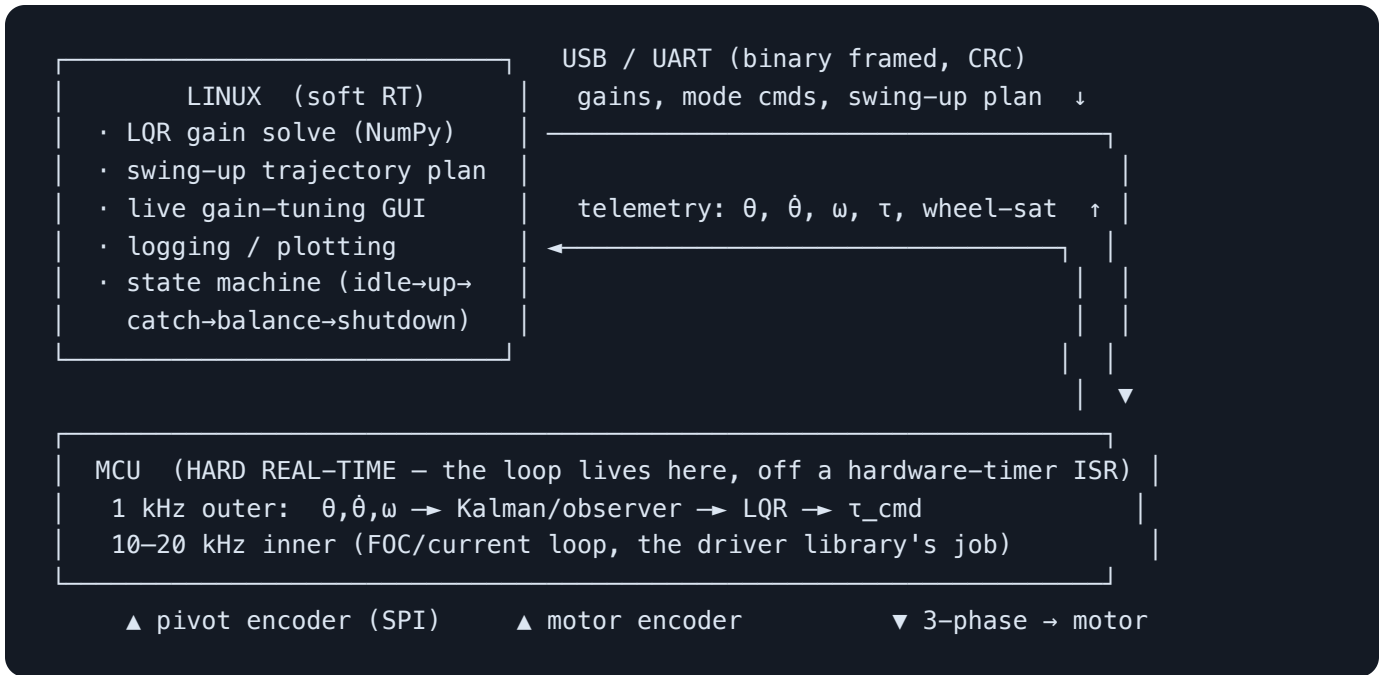
Arduino as a dumb I/O bridge; Linux reads sensors, computes torque, sends it back every cycle. Stock (non-RT) Linux shows **5–20 ms scheduling jitter spikes**, and USB CDC serial adds variable round-trip. A single 20 ms stall grows the tip error by $e^{(8.45 \times 0.020)} \approx 1.18$ — **18% divergence in one hiccup**, ~24% of your entire doubling time. Balance diverges intermittently and you blame your gains for weeks.

BANDWIDTH IS A RED HERRING

A native-USB MCU does 480 Mbit/s. The killer is *latency and jitter*, not throughput. A PREEMPT_RT kernel shrinks OS jitter but does nothing about USB round-trip jitter — it does not rescue this option.

Option C – your split, done RIGHT (YOUR TARGET)

Keep the fast loop on the MCU. Add Linux as a **supervisor** over a binary serial protocol. The link carries **gains / setpoints / modes DOWN** and **telemetry UP** — never the closed-loop torque itself.



If the serial link drops, the MCU keeps balancing on its last-known gains — **graceful degradation**. That property is the whole point. The one discipline: "I'll just close this one loop from Python for now" is Option B in disguise. If you must prototype a controller on Linux, do it only for *swing-up* (forgiving), never inverted balance.

Skip for now: ROS2 / micro-ROS

Great ecosystem, real learning tax, overkill for a single pendulum. The MCU-owns-fast-loop split is identical; ROS just changes the transport.

LOOP-RATE VERDICT

Two nested loops. Inner FOC/current loop **10–20 kHz** — the driver library's job. Outer balance loop **1 kHz on the MCU**. You could get away with 200–500 Hz, but 1 kHz is essentially free on the boards below (FPUs / trig accelerators) and buys comfortable margin against an 82 ms doubling time. Just run 1 kHz.

Sensing – encoder for the arm, and here's exactly why

Direct answer to "encoder or IMU?": **encoder on the pivot for the arm angle**. Your suspicion is right — it's lower-noise, lower-latency, and needs zero fusion for the angle itself. There are two things to sense; they want different parts.

Arm angle θ (balance-critical) – absolute magnetic encoder, SPI

Pick: AS5048A, 14-bit, SPI (~\$15–40 breakout). (AS5047P is an equivalent substitute if cheaper.)

- **Absolute** → true tilt at power-on, no homing, survives resets.
- **SPI** → fast, deterministic, low-jitter — keeps up with 1 kHz cleanly.
- **14-bit** ($0.022^\circ/\text{count}$) → resolution matters most for the *rate* estimate; $\sim 4\times$ cleaner than 12-bit.
- **Avoid AS5600 here.** I2C-only, 12-bit; an I2C stall can freeze your loop mid-balance. Fine for day-one bring-up, migrate off before pushing loop rate.

Mount a **diametrically-magnetized** magnet centered on the pivot shaft end, ~ 1 mm gap. Watch the chip's magnitude/AGC flags during bring-up.

Wheel speed ω (low-stakes, speed only) – you get it for free

With FOC, the driver already has an encoder on the rotor for commutation and streams velocity.

Read ω from the driver. In practice a cheap **AS5600 (~\$5–8) on the motor shaft** — I2C and 12-bit are fine for speed, and it doubles as the FOC commutation sensor. Feeds the $-k\cdot\omega$ wheel-speed regulation term. (Sensorless FOC gets unreliable near zero speed — the swing-up regime — so keep it sensed.)

WHERE YOUR KALMAN FILTER EARNS ITS KEEP

The noisy signal is not the angle — it's velocity, from differentiating a quantized encoder. So: encoder is ground truth for θ (no fusion for the angle); run a Luenberger observer or small Kalman fusing *encoder angle + commanded torque* to produce clean θ and ω . A gyro is the one honest place an IMU helps — as a *secondary* rate input, not your primary angle source (on a fixed pivot the accelerometer's gravity reference is corrupted precisely by the arm's own corrective accelerations). Add an ICM-42688 (~\$10) only if you want the fusion exercise.

SECTION 03

Motor + driver + power

Mental model: your controller commands TORQUE, and torque = current

For any DC/BLDC motor, $\tau = K_t \cdot I$. So the key driver decision: use a driver that closes a **current loop**, not one that just sets voltage/PWM. A plain drone ESC or PWM H-bridge fakes it — at nonzero speed, back-EMF means a given duty produces wildly different current (torque). That points straight at **field-oriented control (FOC)** for a BLDC, or a brushed driver with genuine current sense. Steppers: out (torque falls off with speed, the opposite of what you need). **Direct-drive BLDC "gimbal" motor + FOC is the canonical stack.**

How to judge your "generic motors" (the bench test)

Don't judge torque alone. A reaction-wheel motor lives at a **two-constraint operating point**: deliver your torque ($\sim 0.1 \text{ N}\cdot\text{m}$) *while still spinning fast*, without back-EMF choking it. Measure K_t directly:

1. **From back-EMF ($K_t = K_e$ in SI):** spin the motor, scope one phase, $K_t = V_{\text{bemf,peak}} / \omega_{\text{mech}}$ (pole-pair corrected). Or no-load RPM at known voltage: $K_v \approx \text{rpm}/V$, then $K_t \approx 8.27/K_v$.
2. **Direct:** clamp a lever arm to the shaft resting on a kitchen scale, apply a known current, read $\text{force} \times \text{arm-length} = \text{torque}$, $\div$ current.

3. **Check the operating point, not stall:** confirm (0.1 N·m, several-thousand RPM) sits under $\tau \approx K_t \cdot (V - \omega/K_v)/R$ at your bus voltage.

The recommended motor – and an honest torque correction

Pick: iPower / iFlight GM4108H-120T gimbal motor, ~\$25–45. 8 mm hollow shaft; the canonical RWP motor (the SimpleFOC pendulum does full swing-up + balance on essentially this).

SPEC TRAP

Many listings call it "KV1100" — wrong, that's a different drone 4108. Verified iFlight spec: no-load 513–567 RPM at 20 V (≈ 25 – 28 RPM/V), $11.1\ \Omega$, 1.5 A load. A common shortcut gives $K_t = 8.27/kv \approx 0.31\ \text{N}\cdot\text{m}/\text{A}$, but the datasheet's own load spec (1200–1800 g·cm at 1.5 A = 0.12–0.18 N·m) implies $K_t \approx 0.08$ – $0.12\ \text{N}\cdot\text{m}/\text{A}$ — $\sim 3\times$ lower. Trust the datasheet: 0.1 N·m needs ~ 1 A of the 1.5 A rating (modest headroom, not "4× your sim's authority"). Measure your actual K_t . Catch angle = $\text{asin}(\tau_{\text{max}} / \Sigma m \cdot g \cdot \ell)$: $\sim 10^\circ$ at 0.1 N·m, ~ 15 – 17° at 0.15–0.18 N·m.

The gimbal's one real weakness — **momentum budget**. Low kv $\rightarrow$ low speed ceiling; at 12 V it tops out at a few hundred RPM, so the wheel saturates sooner and swing-up is slower. Mitigation: run **20–24 V, not 12 V** (roughly doubles the ceiling), lean on rim-loading so you hit I_w at lower RPM, accept a gentle multi-pump swing-up.

Driver / brain – two tiers

- **CHEAP TIER (start here):** ST B-G431B-ESC1, ~\$30–40 (DigiKey ~\$38.75). STM32G431 MCU + gate driver + 3-phase stage + onboard current sensing, one board, runs SimpleFOC. The G431 runs the FOC inner loop *and* your 1 kHz balance loop (CORDIC + FPU). This is Option A on one \$40 board. Add an AS5600 (~\$5) on the motor.
- **NICER / ALT BRAIN:** Teensy 4.0 (~\$24) / 4.1 (~\$32, microSD logging) + a SimpleFOC driver (SimpleFOCMini ~\$10, or SimpleFOCShield v3 ~\$30–40). Dev ergonomics, SD logging without serial risk, compute headroom (600 MHz M7). Precompute LQR gains on your laptop, paste the matrix in.
- **UPGRADE PATH (only if you measure saturation):** ODrive S1 (~\$149). Best-in-class torque control, high-speed headroom, proper regen/brake handling for reversal spikes. Prove you

need it first.

If your generics are brushed DC: skip FOC — use a current-sensing H-bridge (Pololu G2 High-Power, ~\$25–35) and close a PI current loop in firmware ($\tau = K_t \cdot I$). **If geared: don't use them for the wheel** — backlash + gearbox friction wreck torque control at an 82 ms doubling time. Direct-drive only.

Power

- **Bench:** adjustable 24–30 V / 5 A lab supply (~\$60–80). Set a current limit while tuning — clamps runaway spikes, saves hardware.
- **Portable:** 6S LiPo (~22–25 V), ~\$30–50. Handles reversal spikes better; no current limit for debugging.
- **Mandatory:** a bulk bus cap (~1000 μ F low-ESR, ~\$2) across the driver DC bus. Reaction-wheel torque reverses constantly, so bus current spikes and can go *negative* (regen) on decel; the cap absorbs it.

SECTION 04

Mechanics / fabrication – your plan is correct

A 3D-printed arm + rim-loaded wheel with metal slugs pressed near the rim, direct-driven by a gimbal motor, is exactly the canonical build. Rim loading is unambiguously right: for fixed added mass, $I_w = m \cdot r^2$ is maximized at the largest radius.

THE CONCEPTUAL ANCHOR

Peak *control torque* comes from the MOTOR ($\tau_{\max}$), independent of I_w . $I_w \cdot \omega$ is *momentum storage*. "Fast + light + rim-loaded" beats "heavy + slow" — you buy storage from ω (no mass cost) and I_w from radius (no gravity-torque cost), whereas piling on wheel mass raises both the pivot load *and* the gravity torque you must fight ($\Sigma m \cdot g \cdot \ell$, your catch-angle denominator) for zero control benefit. **Keep total wheel mass down.** This is the #1 error in these builds.

The wheel and slugs

Print a light spoked/disk wheel in **PETG** (holds press-fits without creep; PLA creeps and is brittle). To hit $I_w \approx 3.8e-4$ you need roughly **120–150 g of metal near $r \approx 0.045$ m** — the metal dominates the inertia, the plastic barely contributes. **Slugs: steel ground dowel pins for v1 (~\$10–20)**. Brass is nicer near the sensor magnet (non-magnetic, same density). Tungsten (~\$30–80) only for a smaller/thinner wheel ($2.3\times$ steel density; slip-fit + epoxy, don't press into plastic).

Slug retention & balancing

Centrifugal force per slug = $m \cdot \omega^2 \cdot r$. At ~ 400 – 500 RPM a 15 g slug at $r=0.045$ pulls under ~ 2 N — press-fit + epoxy is trivially safe. **Make pockets BLIND, closed on the radially-outer side**, so each slug bears in compression against solid plastic, never on friction. **Balance the wheel — do NOT skip**. Even a 1 g offset at $r=0.045$, 500 RPM = ~ 0.12 N at ~ 8 Hz, squarely inside your balance-loop bandwidth. Weigh-match slugs before pressing; if the arm buzzes at a frequency that tracks wheel speed, it's imbalance, not your gains.

Bearings, coupling, frame

- **Direct-drive, NO gearbox — non-negotiable.** Backlash creates a dead-zone the controller can't cross fast enough; gear friction destroys the torque $\leftrightarrow$ current relationship FOC depends on.
- **Don't double-spec bearings.** The wheel rides on the motor's own bearings. Your ball bearings are for the **pivot only**: two **608-2RS** ($8\times 22\times 7$, $\sim \$8$ – 12 /pack) on an **8 mm ground steel shaft** (matches the motor bore — standardize on 8 mm).
- **Rigid, heavy base — mandatory.** A floppy tower lets the pivot sway and your encoder reads base motion as arm angle $\rightarrow$ the loop chases a ghost. Metal post bolted to a heavy wide base (2020 extrusion $\sim \$20$ – 30 , or steel plate).

Starter BOM & phased build path

Lean tier (~\$110–150 + supply/frame you may have)

ITEM	PART	~PRICE
Motor + FOC driver + MCU	iPower GM4108H-120T + B-G431B-ESC1 (brain+driver in one)	\$25–45 + \$30–40
Arm encoder	AS5048A SPI breakout + diametric magnet	\$15–40 + \$3
Wheel/motor encoder	AS5600 (FOC commutation + wheel ω) + magnet	\$5–8 + \$3
Pivot bearings	608-2RS $\times$ 2 (8-pack)	\$8–12
Pivot shaft	8 mm ground steel, 12 in	\$8–15
Rim slugs	1/4 in steel dowel pins (~120–150 g)	\$10–20
Bus cap + fasteners + epoxy	1000 μ F, M3 heat-sets, JB Weld	~\$25
Filament	PETG 1 kg (+ PLA for prototyping)	\$20–25
Frame	2020 extrusion + brackets, or scrap steel/plywood	\$20–30

Nicer tier (add/swap)

- **Brain:** Teensy 4.1 (~\$32, SD logging) + SimpleFOCShield v3 (~\$30–40).
- **Driver (only if you measure saturation):** ODrive S1 (~\$149).
- **Wheel:** tungsten slug kit (~\$30–80) for a compact/thin wheel.
- **Sensing:** ICM-42688 IMU (~\$10) for the gyro-fusion exercise.
- **Power:** 24–30 V / 5 A bench supply (~\$60–80).

Phased build path

- 1 Bench-test the motor first.** Measure K_t / no-load RPM / resistance; confirm the operating point at 20–24 V. Decide: gimbal vs your generics.
 - 2 Build the mechanics.** Print arm + wheel, weigh-match and press slugs, balance the wheel, mount the motor direct-drive, build the rigid base. Nothing electrical yet.
 - 3 Get sensors reading.** Pivot AS5048A (SPI) + motor AS5600; verify magnet AGC flags, log clean θ ; calibrate FOC commutation.
 - 4 Close the balance loop on the MCU (Option A).** Precompute LQR gains in Python, run the outer loop at 1 kHz off a hardware timer, inner FOC 10–20 kHz, add the observer for θ/ω . Catch and hold from near-upright.
 - 5 Add the Linux supervisor (Option C).** Binary serial: gains/modes down, telemetry up. Live tuning, dashboard, LQR solve, the idle→swing-up→catch→balance→shutdown state machine. Never surrender the torque loop to the link.
 - 6 Swing-up last.** Run 20–24 V, accept a gentle multi-pump. The upgrade decision gets made here — if and only if you log real saturation.
-

SECTION 06

Decision points where the recommendation flips

- **Do your generics pass the bench test?** Yes & BLDC → use them with a SimpleFOC driver, skip the gimbal. Brushed → Pololu G2 path. Geared → don't use them for the wheel. Fail the operating-point check → buy the GM4108H-120T.
- **Does swing-up saturate?** Measure-then-decide. Clean at 20–24 V → done, don't spend \$149. Saturating → higher-kv BLDC and/or ODrive S1 and/or 24–36 V.
- **Budget for the brain:** tight → B-G431B-ESC1 does everything on one \$40 board (default). Prefer dev ergonomics/logging → Teensy 4.1 + SimpleFOC driver.
- **How much fusion do you want to write?** Minimum → encoder angle + observer for θ (no IMU). Want the Kalman exercise → add an ICM-42688 gyro as a secondary rate input. Don't

make the IMU primary on a fixed pivot.

- **Compact vs cheap wheel:** steel slugs unless you want small/thin → tungsten (slip-fit + epoxy, not pressed).

VERIFIED SOURCES

Sources

[iFlight GM4108H-120T spec page](#) — no-load 513–567 RPM @ 20 V, 11.1 Ω , load torque 1200–1800 g·cm ($\Rightarrow K_t \approx 0.08\text{--}0.12\text{ N}\cdot\text{m/A}$, $\sim 3\times$ below the theoretical 0.31 figure).

[B-G431B-ESC1 on DigiKey](#) — STM32G431CB + L6387 + FOC + current sensing, $\sim \$38.75$ in stock.

All other prices are approximate 2026 USD and skew low in the source data – ballpark, verify before ordering.

Reaction-Wheel Pendulum · Build & Architecture Guide. Start with Option A; layer Linux on as Option C; never close the balance loop over USB.

Companion doc: "Swinging It Up" (control notes 01).